Computer Science

Portfolio - Period 4

Francisco Pimentel

Student No. 5708915

francisco.ponte.pimentel@nhlstenden.student.com

NHL Stenden University of Applied Sciences

April 2026

Table of Contents

Images Index *3*

Introduction *4*

Computer Science *5*

Week 2 ..... **5**
PageRank 5
Heredity 8
GIRank, Genetic Influence Rank 14
Design Thinking, AI Solutions Brainstorm 15

Week 3 ..... **17**
Crossword 17
AI Flow Schema 23

Week 4 & 5 ..... **25**
Shopping 25
Nim 28
Bug Hunting 33

Week 6 ..... **34**
Traffic 34

Week 7 ..... **37**
Parser 37
Attention 39

Professional Skilss *43*

360º Feedback **43**
Réka Szunyogh-Kocsis, Friend 43
Jeet, Classmate 43
Joana, HighSchool Friend 43
Marta Arrais, Highschool Classmate 44
India, Friend 44
Julian, Classmate 44
Self Reflection 45
Comparing Feedback with Self-Image 45
Learning Points 45

FeedForward, Feed Up and More **46**
Assignment 1, Essay 46
Assignment 2, Pair Assignment 47
Assignment 3, Individual Reflection 47

Self-Reflection, 3rd Period *48*

Self-Reflection, 4th Period *49*

Links *50*

Assignements **50**

Solo Project **50**

Group Project **50**

Images Index

Figure 1 - PageRank | Result 7
Figure 2 - Heredity | Result 1 12
Figure 3 - Heredity | Result 2 13
Figure 4 - Heredity | Result 3 13
Figure 5 - GIRank | Result 15
Figure 6 - Crossword | Result 1 21
Figure 7 - Crossword | Result 2 21
Figure 8 - Crossword | Result 3 22
Figure 9 - AI Flow Schema | Solo Project 23
Figure 10 - Shopping | Result 27
Figure 11 - Nim | Result 32
Figure 12 - Traffic | Result 36
Figure 13 - Parser | Result 38
Figure 14 - Attention | Result | Layer 3, Head 1 42
Figure 15 - Attention | Result | Layer 5, Head 4 42

Introduction

This portfolio documents my work throughout Period 4 of the Computer Science course at NHL Stenden University of Applied Sciences. It covers the assignments completed during the seven weeks of the period, the development of my Solo Project, and my professional development work.

The Computer Science section follows the weekly structure of the course, covering topics across search algorithms, probability models, constraint satisfaction, machine learning, deep learning, and natural language processing. Each entry includes a description of the assignment, the code I implemented, and the results obtained. Where relevant, I also document the reasoning behind technical decisions and the approaches I explored.

The Solo Project, Math Animator, is a tool that transforms linear equations into step-by-step animated videos. It was extended this period to include a natural language input mode, allowing users to describe equations in plain text, which are then converted to mathematical notation using the Groq API with the Llama 3.3 model.

The Professional Skills section includes 360° feedback collected from peers and friends, a self-reflection on my collaboration competence, and the assignments completed as part of the professional development module. Together these document not just what I built this period, but how I worked and what I am taking forward.

Computer Science

Week 2

This week introduces graph-based algorithms and probability models applied to real world problems. The PageRank algorithm is studied and implemented to rank web pages, followed by the Heredity assignment which applies Bayesian probability to genetic inheritance. Finally, GIRank adapts the page rank logic to calculate genetic influence across a family tree.

PageRank

Implementation of the page rank algorithm using 2 approaches: sampling and iterative convergence. The sampling method visits pages randomly according to the transition model, while the iterative method updates rank values until the change between iterations falls below 0.001. Both methods produce consistent results and sum to 1.

```
import os
import random
import re
import sys
DAMPING = 0.85
# number of samples I'll use to estimate PageRank using the sampling method
SAMPLES = 10000
def main() -> None:
    if len(sys.argv) != 2:
        sys.exit("Usage: python pagerank.py corpus")
    corpus = crawl(sys.argv[1])
    ranks = sample_pagerank(corpus, DAMPING, SAMPLES)
    print(f"PageRank Results from Sampling (n = {SAMPLES})")
    for page in sorted(ranks):
        print(f"  {page}: {ranks[page]:.4f}")
    ranks = iterate_pagerank(corpus, DAMPING)
    print(f"PageRank Results from Iteration")
    for page in sorted(ranks):
        print(f"  {page}: {ranks[page]:.4f}")
def crawl(directory):
    """
    Parse a directory of HTML pages and check for links to other pages.
    Return a dictionary where each key is a page, and values are
    a list of all other pages in the corpus that are linked to by the page.
    """
    pages = dict()
    # Extract all links from HTML files
    for filename in os.listdir(directory):
        if not filename.endswith(".html"):
            continue
        with open(os.path.join(directory, filename)) as f:
            contents = f.read()
            links = re.findall(r"<a\s+(?:[^>]*?)href=\"([^\"]*)\"", contents)
            pages[filename] = set(links) - {filename}
```

```
    # Only include links to other pages in the corpus
    for filename in pages:
        pages[filename] = set(
            link for link in pages[filename]
            if link in pages
        )
    return pages
"""
Return a probability distribution over which page to visit next,
given a current page.
With probability `damping_factor`, choose a link at random
linked to by `page`. With probability `1 - damping_factor`, choose
a link at random chosen from all pages in the corpus.
"""
def transition_model(corpus, page, damping_factor) -> dict:

    all_pages = {}
    probability = (1 - damping_factor) / len(corpus)
    for p in corpus:
        all_pages[p] = probability
    if corpus[page]:
        extra = damping_factor / len(corpus[page])

        for link in corpus[page]:
            all_pages[link] += extra
    else:
        for p in corpus[page]:
            all_pages[p] += probability
    return all_pages
"""
Return PageRank values for each page by sampling `n` pages
according to transition model, starting with a page at random.
Return a dictionary where keys are page names, and values are
their estimated PageRank value (a value between 0 and 1). All
PageRank values should sum to 1.
"""
def sample_pagerank(corpus, damping_factor, n) -> dict:
    n_pages = {p: 0 for p in corpus}
    page = random.choice(list(corpus.keys()))
    for i in range(n):
        n_pages[page] +=1
        probs = transition_model(corpus, page, damping_factor)
        next_page = random.choices(
            list(probs.keys()),
            weights = probs.values(),
            k = 1
        )[0]
        page = next_page

    return {p: n_pages[p] / n for p in n_pages}
"""
Return PageRank values for each page by iteratively updating
PageRank values until convergence.
Return a dictionary where keys are page names, and values are
their estimated PageRank value (a value between 0 and 1). All
PageRank values should sum to 1.
"""
def iterate_pagerank(corpus, damping_factor):
```

```
    page_rank = {p: 1 / len(corpus) for p in corpus}
    while True:
        new_page_rank = {}
        for page in corpus:
            soma = 0
            for i in corpus:
                if len(corpus[i]) == 0:
                    soma += page_rank[i] / len(corpus)
                elif page in corpus[i]:
                    soma += page_rank[i] / len(corpus[i])
          new_page_rank[page] = (1 - damping_factor) / len(corpus) + damping_factor * soma
        if all(abs(new_page_rank[p] - page_rank[p]) < 0.001 for p in corpus):
            break
        page_rank = new_page_rank
    return page_rank
if __name__ == "__main__":
    main()
```

```
PROBLEMS  OUTPUT  MEMORY  XRTOS  TERMINAL  DEBUG CONSOLE  PORTS  SERIAL MONITOR  zsh - pagerank

PageRank Results from Sampling (n = 10000)
  1.html: 0.2316
  2.html: 0.4299
  3.html: 0.2143
  4.html: 0.1242
PageRank Results from Iteration
  1.html: 0.2202
  2.html: 0.4289
  3.html: 0.2202
  4.html: 0.1307
$ python3 pagerank.py corpus1
PageRank Results from Sampling (n = 10000)
  bfs.html: 0.1178
  dfs.html: 0.0795
  games.html: 0.2234
  minesweeper.html: 0.1181
  minimax.html: 0.1296
  search.html: 0.2141
  tictactoe.html: 0.1175
PageRank Results from Iteration
  bfs.html: 0.1151
  dfs.html: 0.0806
  games.html: 0.2272
  minesweeper.html: 0.1183
  minimax.html: 0.1305
  search.html: 0.2100
  tictactoe.html: 0.1183
$ python3 pagerank.py corpus2
PageRank Results from Sampling (n = 10000)
  ai.html: 0.1939
  algorithms.html: 0.1062
  c.html: 0.1237
  inference.html: 0.1308
  logic.html: 0.0267
  programming.html: 0.2262
  python.html: 0.1233
  recursion.html: 0.0692
PageRank Results from Iteration
  ai.html: 0.1884
  algorithms.html: 0.1067
  c.html: 0.1243
  inference.html: 0.1291
  logic.html: 0.0264
  programming.html: 0.2293
  python.html: 0.1243
  recursion.html: 0.0716
```

Figure 1 - PageRank | Result

me50 / users / franciscpime / ai50 / projects / 2024 / x / pagerank

#1 submitted a few seconds ago, Friday, April 24, 2026 2:38 PM CEST
check50 11/11 • style50 0.95 • 0 comments
tar.gz • zip

Heredity

A Bayesian probability model that calculates the likelihood of each person having a certain number of copies of a gene, and whether they exhibit the associated trait. The algorithm iterates over all possible combinations of gene distributions and uses joint probability to update each person's probability distribution, which is then normalized to sum to 1.

```
import csv
import itertools
import sys
PROBS = {
    # Unconditional probabilities for having gene
    # Ex: the probability if we know nothing about that person's parents
    #   >>> there's a 1% chance of having 2 copies of the gene
    #   >>> 3% chance of having 1 copy of the gene
    #   >>> 96% chance of having 0 copies of the gene
    "gene": {
        2: 0.01,
        1: 0.03,
        0: 0.96
    },
    "trait": {
        # Probability of trait given two copies of gene
        # So PROBS["trait"][2] is the probability distribution that a person has
        # the trait given that they have two versions of the gene:
        #   >>> 65% chance of exhibiting the trait
        #   >>> 35% chance of not exhibiting the trait
        2: {
            True: 0.65,
            False: 0.35
        },
        # Probability of trait given one copy of gene
        1: {
            True: 0.56,
            False: 0.44
        },
        # Probability of trait given no gene
        # If a person has 0 copies of the gene, they have:
        #   >>> 1% chance of exhibiting the trait
        #   >>> 99% chance of not exhibiting the trait
        0: {
            True: 0.01,
            False: 0.99
        }
    },
    # Mutation probability
    # If a mother has two versions of the gene and passes one on to her child:
```

```
    #   >>> 1% chance it mutates into not being the target gene anymore.
    #
    # If a mother has no versions of the gene, and doesn't pass it:
    #   >>> 1% chance it mutates into being the target gene.
    #
    # It's therefore possible that even if neither parent has any copies
    # of the gene in question, their child might have:
    #   >>> 1 or even 2 copies of the gene
    "mutation": 0.01
}
def main():
    # Check for proper usage
    if len(sys.argv) != 2:
        sys.exit("Usage: python heredity.py data.csv")
    # loads data from a file into a dictionary "people"
    people = load_data(sys.argv[1])
    # Keep track of gene and trait probabilities for each person
    # Ex:
    #   >>> probabilities["Harry"]["gene"][1]
    #       will be the probability that Harry has 1 copy of the gene
    #   >>> probabilities["Lily"]["trait"][False]
    #       will be the probability that Lily does not exhibit the trait
    probabilities = {
        person: {
            "gene": {
                2: 0,
                1: 0,
                0: 0
            },
            "trait": {
                True: 0,
                False: 0
            }
        }
        for person in people
    }
    # Loop over all sets of people who might have the trait
    names = set(people)
    for have_trait in powerset(names):
        # Check if current set of people violates known information
        fails_evidence = any(
            (people[person]["trait"] is not None and
             people[person]["trait"] != (person in have_trait))
            for person in names
        )
        if fails_evidence:
            continue
        # Loop over all sets of people who might have the gene
        for one_gene in powerset(names):
            for two_genes in powerset(names - one_gene):
                # Update probabilities with new joint probability
                p = joint_probability(people, one_gene, two_genes, have_trait)
                update(probabilities, one_gene, two_genes, have_trait, p)
    # Ensure probabilities sum to 1
    normalize(probabilities)
    # Print results
    for person in people:
        print(f"{person}:")
```

```
        for field in probabilities[person]:
            print(f"  {field.capitalize()}:")
            for value in probabilities[person][field]:
                p = probabilities[person][field][value]
                print(f"    {value}: {p:.4f}")
"""
Load gene and trait data from a file into a dictionary.
File assumed to be a CSV containing fields name, mother, father, trait.
mother, father must both be blank, or both be valid names in the CSV.
trait should be 0 or 1 if trait is known, blank otherwise.
"""
def load_data(filename):

    data = dict()
    with open(filename) as f:
        reader = csv.DictReader(f)
        for row in reader:
            name = row["name"]
            data[name] = {
                "name": name,
                "mother": row["mother"] or None,
                "father": row["father"] or None,
                "trait": (True if row["trait"] == "1" else
                          False if row["trait"] == "0" else None)
            }
    return data
def powerset(s):
    """
    Return a list of all possible subsets of set s.
    """
    s = list(s)
    return [
        set(s) for s in itertools.chain.from_iterable(
            itertools.combinations(s, r) for r in range(len(s) + 1)
        )
    ]
"""
Compute and return a joint probability.
The probability returned should be the probability that
    * everyone in set `one_gene` has one copy of the gene, and
    * everyone in set `two_genes` has two copies of the gene, and
    * everyone not in `one_gene` or `two_gene` does not have the gene, and
    * everyone in set `have_trait` has the trait, and
    * everyone not in set` have_trait` does not have the trait.
"""
def joint_probability(people: dict, one_gene: set, two_genes: set, have_trait: set):
    # Joint probability
    prob = 1

    for person in people:
        if person in one_gene:
            num_genes = 1
        elif person in two_genes:
            num_genes = 2
        else:
            num_genes = 0
        if person in have_trait:
            has_trait = True
```

```
        else:
            has_trait = False
        mother = people[person]["mother"]
        father = people[person]["father"]
        if (people[person]["mother"] is None
            and people[person]["father"] is None):
            prob *= PROBS["gene"][num_genes]
            prob *= PROBS["trait"][num_genes][has_trait]
        else:
            if mother in one_gene:
                mum_gene = 1
            elif mother in two_genes:
                mum_gene = 2
            else:
                mum_gene = 0
            if father in one_gene:
                dad_gene = 1
            elif father in two_genes:
                dad_gene = 2
            else:
                dad_gene = 0

            if mum_gene == 1:
                mum_prob = 0.5
            elif mum_gene == 2:
                mum_prob = 1 - PROBS["mutation"]
            else:
                mum_prob = PROBS["mutation"]
            if dad_gene == 1:
                dad_prob = 0.5
            elif dad_gene == 2:
                dad_prob = 1 - PROBS["mutation"]
            else:
                dad_prob = PROBS["mutation"]

            if num_genes == 2:
                comb_prob = mum_prob * dad_prob
            elif num_genes == 1:
                comb_prob = (1-mum_prob) * dad_prob + mum_prob * (1-dad_prob)
            else:
                comb_prob = (1-mum_prob) * (1-dad_prob)
            prob *= comb_prob * PROBS["trait"][num_genes][has_trait]
    return prob
"""
Add to `probabilities` a new joint probability `p`.
Each person should have their "gene" and "trait" distributions updated.
Which value for each distribution is updated depends on whether
the person is in `have_gene` and `have_trait`, respectively.
"""
def update(probabilities, one_gene, two_genes, have_trait, p):
    for person in probabilities:
        if person in one_gene:
            num_genes = 1
        elif person in two_genes:
            num_genes = 2
        else:
            num_genes = 0
        probabilities[person]["gene"][num_genes] += p
```

```
        if person in have_trait:
            has_trait = True
        else:
            has_trait = False
        probabilities[person]["trait"][has_trait] += p

"""
Update `probabilities` such that each probability distribution
is normalized (i.e., sums to 1, with relative proportions the same).
"""
def normalize(probabilities):
    for person in probabilities:
        gene1 = probabilities[person]["gene"][0] / sum(probabilities[person]["gene"].value
s())
        gene2 = probabilities[person]["gene"][1] / sum(probabilities[person]["gene"].value
s())
        gene3 = probabilities[person]["gene"][2] / sum(probabilities[person]["gene"].value
s())
        probabilities[person]["gene"][0] = gene1
        probabilities[person]["gene"][1] = gene2
        probabilities[person]["gene"][2] = gene3
        trait1 = probabilities[person]["trait"][True] / sum(probabilities[person]["trait"]
.values())
        trait2 = probabilities[person]["trait"][False] / sum(probabilities[person]["trait"
].values())

        probabilities[person]["trait"][True] = trait1
        probabilities[person]["trait"][False] = trait2
if __name__ == "__main__":
    main()
```

```
PROBLEMS  OUTPUT  MEMORY  XRTOS  TERMINAL  DEBUG CONSOLE  PORTS  ···   zsh - heredity 2

$ python3 heredity.py data/family0.csv
Harry:
  Gene:
    2: 0.0092
    1: 0.4557
    0: 0.5351
  Trait:
    True: 0.2665
    False: 0.7335
James:
  Gene:
    2: 0.1976
    1: 0.5106
    0: 0.2918
  Trait:
    True: 1.0000
    False: 0.0000
Lily:
  Gene:
    2: 0.0036
    1: 0.0136
    0: 0.9827
  Trait:
    True: 0.0000
    False: 1.0000
```

Figure 2 - Heredity | Result 1

```
$ python3 heredity.py data/family1.csv
Arthur:
  Gene:
    2: 0.0329
    1: 0.1035
    0: 0.8636
  Trait:
    True: 0.0000
    False: 1.0000
Charlie:
  Gene:
    2: 0.0018
    1: 0.1331
    0: 0.8651
  Trait:
    True: 0.0000
    False: 1.0000
Fred:
  Gene:
    2: 0.0065
    1: 0.6486
    0: 0.3449
  Trait:
    True: 1.0000
    False: 0.0000
Ginny:
  Gene:
    2: 0.0027
    1: 0.1805
    0: 0.8168
  Trait:
    True: 0.1110
    False: 0.8890
Molly:
  Gene:
    2: 0.0329
    1: 0.1035
    0: 0.8636
  Trait:
    True: 0.0000
    False: 1.0000
Ron:
  Gene:
    2: 0.0027
    1: 0.1805
    0: 0.8168
  Trait:
    True: 0.1110
    False: 0.8890
```

Figure 3 - Heredity | Result 2

```
PROBLEMS  OUTPUT  MEMORY  XRTOS  TERMINAL  DEBUG CONSOLE  PORTS  SERIAL MONITOR        zsh - heredity 2

$ python3 heredity.py data/family2.csv
Arthur:
  Gene:
    2: 0.0147
    1: 0.0344
    0: 0.9509
  Trait:
    True: 0.0000
    False: 1.0000
Hermione:
  Gene:
    2: 0.0608
    1: 0.1203
    0: 0.8189
  Trait:
    True: 0.0000
    False: 1.0000
Molly:
  Gene:
    2: 0.0404
    1: 0.0744
    0: 0.8852
  Trait:
    True: 0.0768
    False: 0.9232
Ron:
  Gene:
    2: 0.0043
    1: 0.2149
    0: 0.7808
  Trait:
    True: 0.0000
    False: 1.0000
Rose:
  Gene:
    2: 0.0088
    1: 0.7022
    0: 0.2890
  Trait:
    True: 1.0000
    False: 0.0000
```

Figure 4 - Heredity | Result 3

me50 / users / franciscpime / ai50 / projects / 2024 / x / heredity

#1 submitted a few seconds ago, Sunday, May 3, 2026 12:35 PM CEST
check50 16/16 • style50 0.98 • 0 comments
tar.gz • zip

GIRank, Genetic Influence Rank

An iterative algorithm that determines which ancestor in the family tree has the greatest genetic influence on the current generation. Mirrors the PageRank logic: each parent distributes its rank equally among its children and the influence propagates through generations. A damping factor of 0.85 prevents the rank from concentrating at a single node.

```
def girank(family_tree, damping=0.85, iterations=100):
    # Collect all unique people in the tree
    all_people = set()
    for parent, children in family_tree.items():
        all_people.add(parent)
        for child in children:
            all_people.add(child)
    n = len(all_people)
    # Initialize every person with equal rank
    rank = {person: 1 / n for person in all_people}
    # Build a reverse map: for each person, who are their parents?
    parents_of = {person: [] for person in all_people}
    for parent, children in family_tree.items():
        for child in children:
            parents_of[child].append(parent)
    # Iterative GIRank calculation
    for _ in range(iterations):
        new_rank = {}
        for person in all_people:
            # Base rank from damping (like the random surfer in PageRank)
            inherited = (1 - damping) / n
            # Add influence from each parent
            for parent in parents_of[person]:
                num_children = len(family_tree[parent])
                # Parent splits influence equally among all children
                inherited += damping * (rank[parent] / num_children)
            new_rank[person] = inherited
        rank = new_rank
    return rank
def print_girank(rank):
    """Prints the GIRank results sorted from most to least influential."""
    sorted_rank = sorted(rank.items(), key=lambda x: x[1], reverse=True)
    print("\n=== Genetic Influence Rank (GIRank) ===\n")
    for i, (person, score) in enumerate(sorted_rank, 1):
        print(f"  {i}. {person:<10} >> {score:.6f}")
    print()
family_tree = {
    "Alice":   ["Bob", "Charlie"],
    "Bob":     ["David"],
    "Charlie": ["Eve", "Frank"],
    "David":   ["George"],
    "Eve":     ["Hannah"],
    "Frank":   ["Isaac"],
}
rank = girank(family_tree)
print_girank(rank)
```

```
=== Genetic Influence Rank (GIRank) ===

  1. George     >> 0.047993
  2. Isaac      >> 0.039413
  3. Hannah     >> 0.039413
  4. David      >> 0.036854
  5. Frank      >> 0.026760
  6. Eve        >> 0.026760
  7. Charlie    >> 0.023750
  8. Bob        >> 0.023750
  9. Alice      >> 0.016667
```

Figure 5 - GIRank | Result

Design Thinking, AI Solutions Brainstorm

As part of the Solo Project, 10 AI solutions were brainstormed for the defined problem. The 3 most promising ideas were then selected and justified.

10 Brainstormed AI Solutions:

1. AI that interprets natural language ("animate the derivative of x squared") and automatically converts it to the correct terminal input
2. AI that detects errors in the written equation and suggests corrections before generating the animation
3. AI that automatically chooses the most suitable visual style for each type of equation (graph, step-by-step, geometric)
4. AI that automatically generates a voice narration explaining each step of the animation
5. AI that suggests related equations after solving one, creating a learning path
6. AI that adapts the animation speed and complexity based on the user's level (beginner, intermediate, advanced)
7. AI that recognises handwritten equations (from an image) and converts them to text for the terminal
8. AI that detects if an equation has multiple solving methods and asks the user which one they want animated
9. AI that automatically generates similar exercises based on the equation entered, for further practice
10. AI that translates the generated animation's captions and explanations into different languages

Best Ideas and Why

1. **Natural Language to Input:** Instead of writing equations in a specific terminal syntax, the user could simply type "solve and animate the integral of 2x" and the AI converts it to the correct format. This removes the entry barrier for less technical users and makes the tool far more accessible without changing the animation engine itself.
2. **Handwritten Equation Recognition:** Allows the user to take a photo of an exercise from a textbook or whiteboard and the AI extracts the equation and passes it directly to Math Animator. It is a natural extension of the project that bridges the physical world with the tool, something with enormous practical value for students.
3. **Automatic Visual Style Suggestion:** Different equations benefit from different representations, a quadratic equation works better as a graph, an algebraic step-by-step works better as a sequence of operations. An AI that analyses the equation type and automatically selects the clearest visual style would make animations more pedagogical with no extra effort from the user.

Week 3

This week focuses on constraint satisfaction problems and how AI can use structured reasoning to find solutions within defined boundaries. The Crossword assignment applies CSP techniques to fill a puzzle grid using arc consistency and backtracking search. The week also includes the AI Flow Schema for the Solo Project, mapping the full pipeline of the Math Animator tool.

Crossword

A CSP-based crossword puzzle solver that enforces consistency and uses backtracking to assign words to variables. Node consistency is first enforced by removing words that don't match the required length of each slot. Arc consistency is then applied through the AC-3 algorithm, which propagates constraints between overlapping variables. If the problem remains unsolved after constraint propagation, the backtrack function recursively assigns words using the minimum remaining values heuristic and least-constraining value ordering.

```
import sys
from crossword import *
class CrosswordCreator():
    def __init__(self, crossword):
        """
        Create new CSP crossword generate.
        """
        self.crossword = crossword
        self.domains = {
            var: self.crossword.words.copy()
            for var in self.crossword.variables
        }
    def letter_grid(self, assignment):
        """
        Return 2D array representing a given assignment.
        """
        letters = [
            [None for _ in range(self.crossword.width)]
            for _ in range(self.crossword.height)
        ]
        for variable, word in assignment.items():
            direction = variable.direction
            for k in range(len(word)):
                i = variable.i + (k if direction == Variable.DOWN else 0)
                j = variable.j + (k if direction == Variable.ACROSS else 0)
                letters[i][j] = word[k]
        return letters
    def print(self, assignment):
        """
        Print crossword assignment to the terminal.
        """
        letters = self.letter_grid(assignment)
        for i in range(self.crossword.height):
            for j in range(self.crossword.width):
```

```
                if self.crossword.structure[i][j]:
                    print(letters[i][j] or " ", end="")
                else:
                    print("", end="")
            print()
    def save(self, assignment, filename):
        """
        Save crossword assignment to an image file.
        """
        from PIL import Image, ImageDraw, ImageFont
        cell_size = 100
        cell_border = 2
        interior_size = cell_size - 2 * cell_border
        letters = self.letter_grid(assignment)
        # Create a blank canvas
        img = Image.new(
            "RGBA",
            (self.crossword.width * cell_size,
             self.crossword.height * cell_size),
            "black"
        )
        font = ImageFont.truetype("assets/fonts/OpenSans-Regular.ttf", 80)
        draw = ImageDraw.Draw(img)
        for i in range(self.crossword.height):
            for j in range(self.crossword.width):
                rect = [
                    (j * cell_size + cell_border,
                     i * cell_size + cell_border),
                    ((j + 1) * cell_size - cell_border,
                     (i + 1) * cell_size - cell_border)
                ]
                if self.crossword.structure[i][j]:
                    draw.rectangle(rect, fill="white")
                    if letters[i][j]:
                        _, _, w, h = draw.textbbox((0, 0), letters[i][j], font=font)
                        draw.text(
                            (rect[0][0] + ((interior_size - w) / 2),
                             rect[0][1] + ((interior_size - h) / 2) - 10),
                            letters[i][j], fill="black", font=font
                        )
        img.save(filename)
    def solve(self):
        """
        Enforce node and arc consistency, and then solve the CSP.
        """
        self.enforce_node_consistency()
        self.ac3()
        return self.backtrack(dict())
    """
    Update `self.domains` such that each variable is node-consistent.
    (Remove any values that are inconsistent with a variable's unary
        constraints; in this case, the length of the word.)
    """
    def enforce_node_consistency(self):
        for domain in self.domains:
            for word in self.domains[domain].copy():
                if len(word) != domain.length:
                    self.domains[domain].remove(word)
```

```
    """
    Make variable `x` arc consistent with variable `y`.
    To do so, remove values from `self.domains[x]` for which there is no
    possible corresponding value for `y` in `self.domains[y]`.
    Return True if a revision was made to the domain of `x`; return
    False if no revision was made.
    """
    def revise(self, x, y):
        overlap = self.crossword.overlaps[x, y]
        if overlap is None:
            return False
        i, j = overlap
        revised = False
        for word_x in self.domains[x].copy():
            if not any(word_x[i] == word_y[j] for word_y in self.domains[y]):
                self.domains[x].remove(word_x)
                revised = True
        return revised

    """
    Update `self.domains` such that each variable is arc consistent.
    If `arcs` is None, begin with initial list of all arcs in the problem.
    Otherwise, use `arcs` as the initial list of arcs to make consistent.
    Return True if arc consistency is enforced and no domains are empty;
    return False if one or more domains end up empty.
    """
    def ac3(self, arcs=None):

        if arcs is None:
            queue = list(self.crossword.overlaps.keys())
        else:
            queue = list(arcs)
        while queue:
            x, y = queue.pop()
            if self.revise(x, y):
                if not self.domains[x]:
                    return False
                for z in self.crossword.neighbors(x):
                    if z != y:
                        queue.append((z, x))
        return True
    """
    Return True if `assignment` is complete (i.e., assigns a value to each
    crossword variable); return False otherwise.
    """
    def assignment_complete(self, assignment):
        for variable in self.crossword.variables:
            if variable not in assignment:
                return False
        return True
    """
    Return True if `assignment` is consistent (i.e., words fit in crossword
    puzzle without conflicting characters); return False otherwise.
    """
    def consistent(self, assignment):
        if len(assignment.values()) != len(set(assignment.values())):
            return False
        for variable in assignment:
```

```
            if len(assignment[variable]) != variable.length:
                return False
            for neighbor in self.crossword.neighbors(variable):
                if neighbor in assignment:
                    i, j = self.crossword.overlaps[variable, neighbor]
                    if assignment[variable][i] != assignment[neighbor][j]:
                        return False
        return True

    """
    Return a list of values in the domain of `var`, in order by
    the number of values they rule out for neighboring variables.
    The first value in the list, for example, should be the one
    that rules out the fewest values among the neighbors of `var`.
    """
    def order_domain_values(self, var, assignment):
        counts = {}
        for word in self.domains[var]:
            n = 0
            for neighbor in self.crossword.neighbors(var):
                if neighbor not in assignment:
                    overlap = self.crossword.overlaps[var, neighbor]
                    if overlap:
                        i, j = overlap
                        for neighbor_word in self.domains[neighbor]:
                            if word[i] != neighbor_word[j]:
                                n += 1
            counts[word] = n
        return sorted(self.domains[var], key=lambda word: counts[word])
    """
    Return an unassigned variable not already part of `assignment`.
    Choose the variable with the minimum number of remaining values
    in its domain. If there is a tie, choose the variable with the highest
    degree. If there is a tie, any of the tied variables are acceptable
    return values.
    """
    def select_unassigned_variable(self, assignment):
        unassigned = [v for v in self.crossword.variables if v not in assignment]
        return sorted(unassigned, key=lambda v: (len(self.domains[v]), -
len(self.crossword.neighbors(v))))[0]
    """
    Using Backtracking Search, take as input a partial assignment for the
    crossword and return a complete assignment if possible to do so.
    `assignment` is a mapping from variables (keys) to words (values).
    If no assignment is possible, return None.
    """
    def backtrack(self, assignment):
        if self.assignment_complete(assignment):
            return assignment
        var = self.select_unassigned_variable(assignment)
        for value in self.order_domain_values(var, assignment):
            assignment[var] = value
            if self.consistent(assignment):
                result = self.backtrack(assignment)
                if result is not None:
                    return result
            del assignment[var]
        return None
```

```
def main():
    # Check usage
    if len(sys.argv) not in [3, 4]:
        sys.exit("Usage: python generate.py structure words [output]")
    # Parse command-line arguments
    structure = sys.argv[1]
    words = sys.argv[2]
    output = sys.argv[3] if len(sys.argv) == 4 else None
    # Generate crossword
    crossword = Crossword(structure, words)
    creator = CrosswordCreator(crossword)
    assignment = creator.solve()
    # Print result
    if assignment is None:
        print("No solution.")
    else:
        creator.print(assignment)
        if output:
            creator.save(assignment, output)
if __name__ == "__main__":
    main()
```

```
$ python3 generate.py data/structure0.txt data/words0.txt output.png
 SIX
E F
V I
E V
 NINE
$ python3 generate.py data/structure1.txt data/words0.txt output.png
No solution.
$ python3 generate.py data/structure2.txt data/words0.txt output.png
No solution.
```

Figure 6 - Crossword | Result 1

```
$ python3 generate.py data/structure0.txt data/words1.txt output.png
 BIT
A L
Y I
E N
 SINE
$ python3 generate.py data/structure1.txt data/words1.txt output.png
    M   R
 INTELLIGENCE
N   N   S
F  LOGIC  O
E   M   L
R  SEARCH V
    X   E
$ python3 generate.py data/structure2.txt data/words1.txt output.png
    C
LOSS R
I TRUE
N A A
E R T
 BIT E
```

Figure 7 - Crossword | Result 2

```
$ python3 generate.py data/structure0.txt data/words2.txt output.png
 SIR
 A A
 L I
 E D
 SAVE
$ python3 generate.py data/structure1.txt data/words2.txt output.png
    B   M
 CONSIDERABLE
 H   N   E
 I HONEY T
 E   A   I
 F ATTACH N
    H   G
$ python3 generate.py data/structure2.txt data/words2.txt output.png
    F
SHOE R
P LIFE
I I N
N T C
 PIE H
```

Figure 8 - Crossword | Result 3

me50 / users / franciscpime / ai50 / projects / 2024 / x / crossword

#1 submitted a few seconds ago, Thursday, May 21, 2026 10:38 AM CEST
check50 25/25 • style50 0.98 • 0 comments
tar.gz • zip

AI Flow Schema

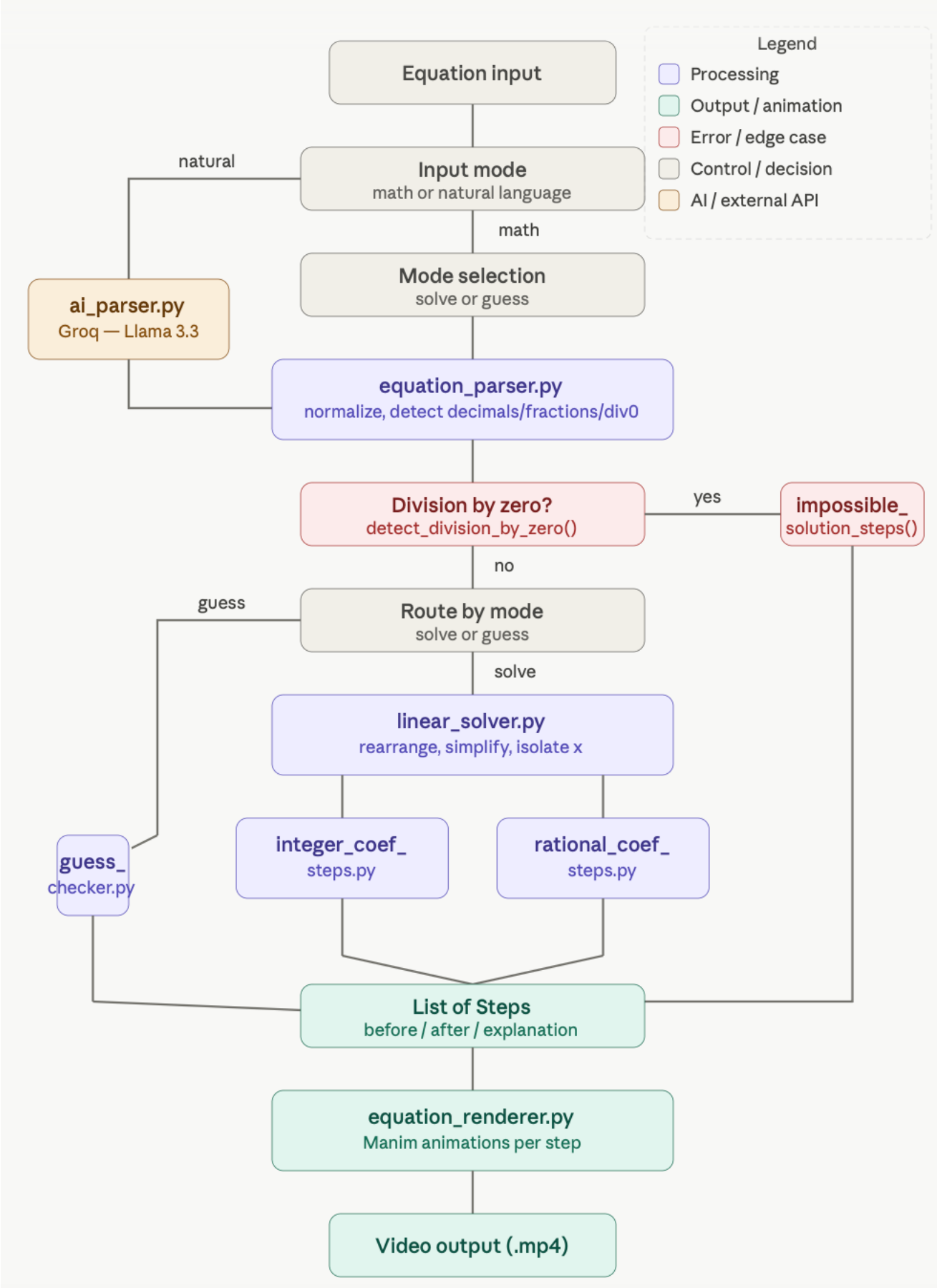


Figure 9 - AI Flow Schema | Solo Project

Math Animator is a tool that transforms linear equations into animated videos that show the step-by-step solution.

The user starts by entering the equation and choosing the input mode: mathematical or natural language. In natural language mode, the equation is described in running text (for example, "two x plus three equals seven") and sent to the Groq API, which uses the Llama 3.3 model to automatically convert it into standard mathematical notation (2x + 3 = 7). This is the AI component of the project.

Next, the user chooses the solution mode: solve or guess. In solve mode, the program automatically solves the equation and generates the complete animation until it reaches the value of x. In guess mode, the user enters a value for x and the program substitutes it into the equation, simplifying both sides step by step to check if it is correct or not.

Regardless of the mode, the equation goes through the parser, which normalizes the expression, detects decimals, undecomposed fractions, and divisions by zero. If there is a division by zero, the flow is interrupted and the animation shows the lateral limits and the impossible solution conclusion.

Otherwise, the solver processes the equation, rearranging terms, simplifying, and isolating x, and generates a list of steps, each with a before state, an after state, and an optional explanation. This list is passed to the Manim renderer, which produces the final video.

Week 4 & 5

These two weeks focus on machine learning, both supervised and reinforcement. The Shopping assignment applies a k-nearest neighbours classifier to predict online purchase intent from user session data. The Nim assignment uses Q-learning to train an AI agent to play the game optimally through self-play. The week also includes a Bug Hunting exercise that requires identifying and explaining logic errors in a BFS implementation.

Shopping

A supervised learning model that predicts whether an online shopper will complete a purchase based on session behaviour data. The dataset is loaded from a CSV file and each row is converted into a feature vector with 17 attributes, including page visit counts, duration times, bounce and exit rates, and contextual flags such as weekend and visitor type. A k-nearest neighbour classifier with k=1 is trained on 60% of the data and evaluated on the remaining 40%, reporting sensitivity (true positive rate) and specificity (true negative rate).

```
import csv
import sys
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
TEST_SIZE = 0.4
def main():
    # Check command-line arguments
    if len(sys.argv) != 2:
        sys.exit("Usage: python shopping.py data")
    # Load data from spreadsheet and split into train and test sets
    evidence, labels = load_data(sys.argv[1])
    X_train, X_test, y_train, y_test = train_test_split(
        evidence, labels, test_size=TEST_SIZE
    )
    # Train model and make predictions
    model = train_model(X_train, y_train)
    predictions = model.predict(X_test)
    sensitivity, specificity = evaluate(y_test, predictions)
    # Print results
    print(f"Correct: {(y_test == predictions).sum()}")
    print(f"Incorrect: {(y_test != predictions).sum()}")
    print(f"True Positive Rate: {100 * sensitivity:.2f}%")
    print(f"True Negative Rate: {100 * specificity:.2f}%")
"""
Load shopping data from a CSV file `filename` and convert into a list of
evidence lists and a list of labels. Return a tuple (evidence, labels).
evidence should be a list of lists, where each list contains the
following values, in order:
    - Administrative, an integer
    - Administrative_Duration, a floating point number
    - Informational, an integer
    - Informational_Duration, a floating point number
```

```
    - ProductRelated, an integer
    - ProductRelated_Duration, a floating point number
    - BounceRates, a floating point number
    - ExitRates, a floating point number
    - PageValues, a floating point number
    - SpecialDay, a floating point number
    - Month, an index from 0 (January) to 11 (December)
    - OperatingSystems, an integer
    - Browser, an integer
    - Region, an integer
    - TrafficType, an integer
    - VisitorType, an integer 0 (not returning) or 1 (returning)
    - Weekend, an integer 0 (if false) or 1 (if true)
labels should be the corresponding list of labels, where each label
is 1 if Revenue is true, and 0 otherwise.
"""
def load_data(filename):

    evidence = []
    labels = []
    months = {"Jan": 0, "Feb": 1, "Mar": 2, "Apr": 3, "May": 4, "June": 5,
              "Jul": 6, "Aug": 7, "Sep": 8, "Oct": 9, "Nov": 10, "Dec": 11}
    with open(filename) as f:
        reader = csv.DictReader(f)
        for row in reader:
            evidence.append([
                int(row["Administrative"]),
                float(row["Administrative_Duration"]),
                int(row["Informational"]),
                float(row["Informational_Duration"]),
                int(row["ProductRelated"]),
                float(row["ProductRelated_Duration"]),
                float(row["BounceRates"]),
                float(row["ExitRates"]),
                float(row["PageValues"]),
                float(row["SpecialDay"]),
                months[row["Month"]],
                int(row["OperatingSystems"]),
                int(row["Browser"]),
                int(row["Region"]),
                int(row["TrafficType"]),
                1 if row["VisitorType"] == "Returning_Visitor" else 0,
                1 if row["Weekend"] == "TRUE" else 0
            ])
            if row["Revenue"] == "TRUE":
                labels.append(1)
            else:
                labels.append(0)
    return (evidence, labels)

"""
Given a list of evidence lists and a list of labels, return a
fitted k-nearest neighbor model (k=1) trained on the data.
"""
def train_model(evidence: list, labels: list):
    classifier = KNeighborsClassifier(n_neighbors = 1)
    classifier.fit(evidence, labels)
    return classifier
```

```
"""
Given a list of actual labels and a list of predicted labels,
return a tuple (sensitivity, specificity).
Assume each label is either a 1 (positive) or 0 (negative).
`sensitivity` should be a floating-point value from 0 to 1
representing the "true positive rate": the proportion of
actual positive labels that were accurately identified.
`specificity` should be a floating-point value from 0 to 1
representing the "true negative rate": the proportion of
actual negative labels that were accurately identified.
"""
def evaluate(labels, predictions):

    tp = 0
    fn = 0
    tn = 0
    fp = 0
    for label, prediction in zip(labels, predictions):
        if label == 1 and prediction == 1:
            tp += 1
        elif label == 1 and prediction == 0:
            fn += 1
        elif label == 0 and prediction == 0:
            tn += 1
        elif label == 0 and prediction == 1:
            fp += 1

    sensitivity = tp / (tp + fn)
    specificity = tn / (tn + fp)
    return (sensitivity, specificity)
if __name__ == "__main__":
    main()
```



Figure 10 - Shopping | Result

me50 / users / franciscpime / ai50 / projects / 2024 / x / shopping

#1 submitted 12 hours ago, Tuesday, May 26, 2026 11:09 AM CEST
No results
tar.gz · zip

Nim

A Q-learning agent trained to play the game of Nim through repeated self-play. The agent maintains a dictionary of Q-values mapping state-action pairs to estimated rewards. During training, after each move the Q-value is updated using the formula Q(s,a) <-- old + α * (reward + future - old), where alpha is the learning rate and future is the best estimated reward from the resulting state. An epsilon-greedy strategy balances exploration and exploitation during training. After training, the agent plays deterministically by always choosing the action with the highest Q-value.

```
import math
import random
import time
class Nim():
    def __init__(self, initial=[1, 3, 5, 7]):
        """
        Initialize game board.
        Each game board has
            - `piles`: a list of how many elements remain in each pile
            - `player`: 0 or 1 to indicate which player's turn
            - `winner`: None, 0, or 1 to indicate who the winner is
        """
        self.piles = initial.copy()
        self.player = 0
        self.winner = None
    @classmethod
    def available_actions(cls, piles):
        """
        Nim.available_actions(piles) takes a `piles` list as input
        and returns all of the available actions `(i, j)` in that state.
        Action `(i, j)` represents the action of removing `j` items
        from pile `i` (where piles are 0-indexed).
        """
        actions = set()
        for i, pile in enumerate(piles):
            for j in range(1, pile + 1):
                actions.add((i, j))
        return actions
    @classmethod
    def other_player(cls, player):
        """
        Nim.other_player(player) returns the player that is not
        `player`. Assumes `player` is either 0 or 1.
        """
        return 0 if player == 1 else 1
    def switch_player(self):
        """
        Switch the current player to the other player.
        """
        self.player = Nim.other_player(self.player)
    def move(self, action):
        """
        Make the move `action` for the current player.
        `action` must be a tuple `(i, j)`.
        """
        pile, count = action
```

```
        # Check for errors
        if self.winner is not None:
            raise Exception("Game already won")
        elif pile < 0 or pile >= len(self.piles):
            raise Exception("Invalid pile")
        elif count < 1 or count > self.piles[pile]:
            raise Exception("Invalid number of objects")
        # Update pile
        self.piles[pile] -= count
        self.switch_player()
        # Check for a winner
        if all(pile == 0 for pile in self.piles):
            self.winner = self.player
class NimAI():
    def __init__(self, alpha=0.5, epsilon=0.1):
        """
        Initialize AI with an empty Q-learning dictionary,
        an alpha (learning) rate, and an epsilon rate.
        The Q-learning dictionary maps `(state, action)`
        pairs to a Q-value (a number).
         - `state` is a tuple of remaining piles, e.g. (1, 1, 4, 4)
         - `action` is a tuple `(i, j)` for an action
        """
        self.q = dict()
        self.alpha = alpha
        self.epsilon = epsilon
    def update(self, old_state, action, new_state, reward):
        """
        Update Q-learning model, given an old state, an action taken
        in that state, a new resulting state, and the reward received
        from taking that action.
        """
        old = self.get_q_value(old_state, action)
        best_future = self.best_future_reward(new_state)
        self.update_q_value(old_state, action, old, reward, best_future)
    """
    Return the Q-value for the state `state` and the action `action`.
    If no Q-value exists yet in `self.q`, return 0.
    """
    def get_q_value(self, state, action):
        return self.q.get((tuple(state), action), 0)
    """
    Update the Q-value for the state `state` and the action `action`
    given the previous Q-value `old_q`, a current reward `reward`,
    and an estiamte of future rewards `future_rewards`.
    Use the formula:
    Q(s, a) <- old value estimate
                + alpha * (new value estimate - old value estimate)
    where `old value estimate` is the previous Q-value,
    `alpha` is the learning rate, and `new value estimate`
    is the sum of the current reward and estimated future rewards.
    """
    def update_q_value(self, state, action, old_q, reward, future_rewards):
        estimate = reward + future_rewards
        result = old_q + self.alpha * (estimate - old_q)
        self.q[(tuple(state), action)] = result
    """
    Given a state `state`, consider all possible `(state, action)`
```

```
    pairs available in that state and return the maximum of all
    of their Q-values.
    Use 0 as the Q-value if a `(state, action)` pair has no
    Q-value in `self.q`. If there are no available actions in
    `state`, return 0.
    """
    def best_future_reward(self, state):
        actions = Nim.available_actions(state)
        q_values = []
        if not actions:
            return 0

        for action in actions:
            q_value = self.get_q_value(state, action)
            q_values.append(q_value)

        return max(q_values)
    """
    Given a state `state`, return an action `(i, j)` to take.
    If `epsilon` is `False`, then return the best action
    available in the state (the one with the highest Q-value,
    using 0 for pairs that have no Q-values).
    If `epsilon` is `True`, then with probability
    `self.epsilon` choose a random available action,
    otherwise choose the best action available.
    If multiple actions have the same Q-value, any of those
    options is an acceptable return value.
    """
    def choose_action(self, state, epsilon=True):
        actions = Nim.available_actions(state)
        if not epsilon:
            return max(actions, key=lambda action: self.get_q_value(state, action))
        else:
            number = random.random()
            if number < self.epsilon:
                return random.choice(list(actions))
            else:
                return max(actions, key=lambda action: self.get_q_value(state, action))
def train(n):
    """
    Train an AI by playing `n` games against itself.
    """
    player = NimAI()
    # Play n games
    for i in range(n):
        print(f"Playing training game {i + 1}")
        game = Nim()
        # Keep track of last move made by either player
        last = {
            0: {"state": None, "action": None},
            1: {"state": None, "action": None}
        }
        # Game loop
        while True:
            # Keep track of current state and action
            state = game.piles.copy()
            action = player.choose_action(game.piles)
            # Keep track of last state and action
```

```
            last[game.player]["state"] = state
            last[game.player]["action"] = action
            # Make move
            game.move(action)
            new_state = game.piles.copy()
            # When game is over, update Q values with rewards
            if game.winner is not None:
                player.update(state, action, new_state, -1)
                player.update(
                    last[game.player]["state"],
                    last[game.player]["action"],
                    new_state,
                    1
                )
                break
            # If game is continuing, no rewards yet
            elif last[game.player]["state"] is not None:
                player.update(
                    last[game.player]["state"],
                    last[game.player]["action"],
                    new_state,
                    0
                )
    print("Done training")
    # Return the trained AI
    return player
def play(ai, human_player=None):
    """
    Play human game against the AI.
    `human_player` can be set to 0 or 1 to specify whether
    human player moves first or second.
    """
    # If no player order set, choose human's order randomly
    if human_player is None:
        human_player = random.randint(0, 1)
    # Create new game
    game = Nim()
    # Game loop
    while True:
        # Print contents of piles
        print()
        print("Piles:")
        for i, pile in enumerate(game.piles):
            print(f"Pile {i}: {pile}")
        print()
        # Compute available actions
        available_actions = Nim.available_actions(game.piles)
        time.sleep(1)
        # Let human make a move
        if game.player == human_player:
            print("Your Turn")
            while True:
                pile = int(input("Choose Pile: "))
                count = int(input("Choose Count: "))
                if (pile, count) in available_actions:
                    break
                print("Invalid move, try again.")
        # Have AI make a move
```

```
        else:
            print("AI's Turn")
            pile, count = ai.choose_action(game.piles, epsilon=False)
            print(f"AI chose to take {count} from pile {pile}.")
        # Make move
        game.move((pile, count))
        # Check for winner
        if game.winner is not None:
            print()
            print("GAME OVER")
            winner = "Human" if game.winner == human_player else "AI"
            print(f"Winner is {winner}")
            return
```



Figure 11 - Nim | Result

me50 / users / franciscpime / ai50 / projects / 2024 / x / nim

#1 submitted a minute ago, Monday, June 1, 2026 3:18 PM CEST
check50 14/14 • style50 0.99 • 0 comments
tar.gz • zip

Bug Hunting

```
from collections import deque
class Node:
    def __init__(self, state, parent=None):
        self.state = state
        elf.parent = parent

    def bfs(start, goal, neighbors_fn):
        frontier = deque()
        frontier.append(start)
        explored = set()
        while len(frontier) > 0:
            current = frontier.popleft()
            if current.state == goal:
                path = []
                while current is not None:
                    path.append(current)
                    current = current.parent

                return path.reverse()

            explored.add(current.state)

            for neighbor in neighbors_fn(current.state):
                if neighbor not in explored:
                    child = Node(neighbor, current)
                    frontier.append(child)
        return None
```

What is worng?

The code has two main issues. The first is a subtle but critical bug: "return path.reverse()". Python's "list.reverse()" reverses the list in place and returns "None", meaning the function will always return "None" even when the valid path is found. The fix is simple, call "path.reverse()" on its own line, then "return path" separately.

The second issue is that visited states are marked as explored too late. States are only added to the "explored" set after being popped from the frontier, which means the same state can be added to the queue multiple times before it is ever visited. This leads to redundant processing and can significantly slow down the search. The fix is to mark a state has explored at the moment it is added to the frontier, not when it is removed.

Week 6

This week focuses on deep learning applied to computer vision. The Traffic assignment uses a convolutional neural network to classify road signs from the GTSRB dataset, which contains images across 43 categories. The model is built and trained using TensorFlow and Keras, combining feature extraction through convolutional layers with classification through dense layers.

Traffic

A convolutional neural network that classifies road sign images into 43 categories. Images are loaded from a structured directory, resized to 30x30 pixels using OpenCV, and normalised into numpy arrays. The model architecture consists of two convolutional layers each followed by max pooling, a flatten layer, a dense hidden layer with 128 units and ReLU activation, a dropout layer with rate 0.5 to reduce overfitting, and a final softmax output layer with one unit per category. The model is compiled with the Adam optimiser and categorical cross-entropy loss, and trained over 10 epochs, reaching approximately 95% accuracy on the test set.

```
import cv2
import numpy as np
import os
import sys
import tensorflow as tf
from sklearn.model_selection import train_test_split
EPOCHS = 10
IMG_WIDTH = 30
IMG_HEIGHT = 30
NUM_CATEGORIES = 43
TEST_SIZE = 0.4
def main():
    # Check command-line arguments
    if len(sys.argv) not in [2, 3]:
        sys.exit("Usage: python traffic.py data_directory [model.h5]")
    # Get image arrays and labels for all image files
    images, labels = load_data(sys.argv[1])
    # Split data into training and testing sets
    labels = tf.keras.utils.to_categorical(labels)
    x_train, x_test, y_train, y_test = train_test_split(
        np.array(images), np.array(labels), test_size=TEST_SIZE
    )
    # Get a compiled neural network
    model = get_model()
    # Fit model on training data
    model.fit(x_train, y_train, epochs=EPOCHS)
    # Evaluate neural network performance
    model.evaluate(x_test,  y_test, verbose=2)
    # Save model to file
    if len(sys.argv) == 3:
        filename = sys.argv[2]
        model.save(filename)
```

```
        print(f"Model saved to {filename}.")
"""
Load image data from directory `data_dir`.
Assume `data_dir` has one directory named after each category, numbered
0 through NUM_CATEGORIES - 1. Inside each category directory will be some
number of image files.
Return tuple `(images, labels)`. `images` should be a list of all
of the images in the data directory, where each image is formatted as a
numpy ndarray with dimensions IMG_WIDTH x IMG_HEIGHT x 3. `labels` should
be a list of integer labels, representing the categories for each of the
corresponding `images`.
"""
def load_data(data_dir):

    images = []
    labels = []
    for categorie in range(NUM_CATEGORIES):
        path =  os.path.join(data_dir, str(categorie))
        for filename in os.listdir(path):
            filepath = os.path.join(path, filename)
            image = cv2.imread(filepath)
            image = cv2.resize(image, (IMG_WIDTH, IMG_HEIGHT))
            images.append(image)
            labels.append(categorie)

    return (images, labels)
"""
Returns a compiled convolutional neural network model. Assume that the
`input_shape` of the first layer is `(IMG_WIDTH, IMG_HEIGHT, 3)`.
The output layer should have `NUM_CATEGORIES` units, one for each category.
"""
def get_model():
    model = tf.keras.models.Sequential([
        tf.keras.layers.Conv2D(32, (3, 3), activation="relu", input_shape=(IMG_WIDTH, IMG_
HEIGHT, 3)),
        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
        tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation="relu"),
        tf.keras.layers.Dropout(0.5),
        tf.keras.layers.Dense(NUM_CATEGORIES, activation="softmax")
    ])
    model.compile(
        optimizer = "adam",
        loss = "categorical_crossentropy",
        metrics = ["accuracy"]
    )
    return model
if __name__ == "__main__":
    main()
```

```
/Users/franciscopontepimentel/Desktop/University/CS_4th_Period/traffic/venv311/lib/python3.11/site-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.1585 - loss: 3.7301
Epoch 2/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.4319 - loss: 1.9784
Epoch 3/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.5935 - loss: 1.3219
Epoch 4/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.6847 - loss: 0.9903
Epoch 5/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.7654 - loss: 0.7509
Epoch 6/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.8134 - loss: 0.6029
Epoch 7/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.8432 - loss: 0.5092
Epoch 8/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.8719 - loss: 0.4174
Epoch 9/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.8822 - loss: 0.4071
Epoch 10/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.8997 - loss: 0.3245
333/333 - 1s - 2ms/step - accuracy: 0.9501 - loss: 0.1853
```

Figure 12 - Traffic | Result

me50 / users / franciscpime / ai50 / projects / 2024 / x / traffic

#1 submitted a few seconds ago, Monday, June 1, 2026 9:30 PM CEST
check50 7/7 • style50 0.93 • 0 comments
tar.gz • zip

Week 7

The final week covers natural language processing, with two assignments exploring how machines parse and understand text. The Parser assignment uses a context-free grammar to analyse sentence structure and extract noun phrase chunks. The Attention assignment loads a pre-trained BERT model to perform masked language modelling and visualises the self-attention weights across layers and heads, revealing how the model relates tokens to one another.

Parser

A natural language parser built using NLTK and a hand-crafted context-free grammar. The grammar defines terminal symbols for parts of speech and nonterminal rules for sentence, noun phrase, verb phrase, prepositional phrase, and adjective phrase structures. The parser uses a chart parsing algorithm to generate all valid parse trees for a given sentence. A preprocessing step tokenises the input and removes non-alphabetic tokens. The np_chunk function then traverses each tree to extract noun phrase chunks, defined as NP subtrees that contain no nested NP subtrees.

```
import nltk
import sys
TERMINALS = """
Adj -> "country" | "dreadful" | "enigmatical" | "little" | "moist" | "red"
Adv -> "down" | "here" | "never"
Conj -> "and" | "until"
Det -> "a" | "an" | "his" | "my" | "the"
N -> "armchair" | "companion" | "day" | "door" | "hand" | "he" | "himself"
N -> "holmes" | "home" | "i" | "mess" | "paint" | "palm" | "pipe" | "she"
N -> "smile" | "thursday" | "walk" | "we" | "word"
P -> "at" | "before" | "in" | "of" | "on" | "to"
V -> "arrived" | "came" | "chuckled" | "had" | "lit" | "said" | "sat"
V -> "smiled" | "tell" | "were"
"""
NONTERMINALS = """
S -> NP VP | S Conj S
NP -> N | Det N | Det AdjP N | NP PP
VP -> V | V NP | V PP | V NP PP | VP Adv | Adv VP | VP Conj VP
PP -> P NP
AdjP -> Adj | Adj AdjP
"""
grammar = nltk.CFG.fromstring(NONTERMINALS + TERMINALS)
parser = nltk.ChartParser(grammar)
def main():
    # If filename specified, read sentence from file
    if len(sys.argv) == 2:
        with open(sys.argv[1]) as f:
            s = f.read()
    # Otherwise, get sentence as input
    else:
        s = input("Sentence: ")
    # Convert input into list of words
```

```
    s = preprocess(s)
    # Attempt to parse sentence
    try:
        trees = list(parser.parse(s))
    except ValueError as e:
        print(e)
        return
    if not trees:
        print("Could not parse sentence.")
        return
    # Print each tree with noun phrase chunks
    for tree in trees:
        tree.pretty_print()
        print("Noun Phrase Chunks")
        for np in np_chunk(tree):
            print(" ".join(np.flatten()))
"""
Convert `sentence` to a list of its words.
Pre-process sentence by converting all characters to lowercase
and removing any word that does not contain at least one alphabetic
character.
"""
def preprocess(sentence):

    phrase = nltk.word_tokenize(sentence)
    return [token.lower() for token in phrase if any(c.isalpha() for c in token)]
"""
Return a list of all noun phrase chunks in the sentence tree.
A noun phrase chunk is defined as any subtree of the sentence
whose label is "NP" that does not itself contain any other
noun phrases as subtrees.
"""
def np_chunk(tree):

    chunks = []
    for subtree in tree.subtrees():
        if subtree.label() == "NP":
            if not any(s.label() == "NP" for s in subtree.subtrees() if s != subtree):
                chunks.append(subtree)
    return chunks
if __name__ == "__main__":
    main()
```

```
$ python3 parser.py
Sentence: Holmes sat
          S
      ____|___
     NP       VP
     |        |
     N        V
     |        |
holmes      sat

Noun Phrase Chunks
holmes
```

Figure 13 - Parser | Result

me50 / users / franciscpime / ai50 / projects / 2024 / x / parser

#1 submitted 11 minutes ago, Tuesday, June 2, 2026 2:10 PM CEST
check50 10/10 • style50 0.97 • 0 comments
tar.gz • zip

Attention

A masked language model analyser built on top of the pre-trained BERT (bert-base-uncased) model. The program takes a sentence containing a [MASK] token, uses BERT to predict the top 3 most likely replacements, and generates attention diagrams for every layer and head in the model. Each diagram visualises the self-attention weights as a grid where lighter cells indicate stronger attention between token pairs, allowing patterns in how BERT processes language to be observed and interpreted.

```
import sys
import tensorflow as tf
from PIL import Image, ImageDraw, ImageFont
from transformers import AutoTokenizer, TFBertForMaskedLM
# Pre-trained masked language model
MODEL = "bert-base-uncased"
# Number of predictions to generate
K = 3
# Constants for generating attention diagrams
FONT = ImageFont.truetype("assets/fonts/OpenSans-Regular.ttf", 28)
GRID_SIZE = 40
PIXELS_PER_WORD = 200
def main():
    text = input("Text: ")
    # Tokenize input
    tokenizer = AutoTokenizer.from_pretrained(MODEL)
    inputs = tokenizer(text, return_tensors="tf")
    mask_token_index = get_mask_token_index(tokenizer.mask_token_id, inputs)
    if mask_token_index is None:
        sys.exit(f"Input must include mask token {tokenizer.mask_token}.")
    # Use model to process input
    model = TFBertForMaskedLM.from_pretrained(MODEL)
    result = model(**inputs, output_attentions=True)
    # Generate predictions
    mask_token_logits = result.logits[0, mask_token_index]
    top_tokens = tf.math.top_k(mask_token_logits, K).indices.numpy()
    for token in top_tokens:
        print(text.replace(tokenizer.mask_token, tokenizer.decode([token])))
    # Visualize attentions
    visualize_attentions(inputs.tokens(), result.attentions)
"""
Return the index of the token with the specified `mask_token_id`, or
`None` if not present in the `inputs`.
"""
def get_mask_token_index(mask_token_id, inputs):
```

```
    tokens = inputs["input_ids"][0]
    for i, token in enumerate(tokens):
        if token == mask_token_id:
            return i

    return None
"""
Return a tuple of three integers representing a shade of gray for the
given `attention_score`. Each value should be in the range [0, 255].
"""
def get_color_for_attention_score(attention_score):

    value = int(attention_score * 255)
    return (value, value, value)
"""
Produce a graphical representation of self-attention scores.
For each attention layer, one diagram should be generated for each
attention head in the layer. Each diagram should include the list of
`tokens` in the sentence. The filename for each diagram should
include both the layer number (starting count from 1) and head number
(starting count from 1).
"""
def visualize_attentions(tokens, attentions):

    for i in range(len(attentions)):
        for k in range(len(attentions[0][0])):
            generate_diagram(
                i + 1,
                k + 1,
                tokens,
                attentions[i][0][k]
            )
def generate_diagram(layer_number, head_number, tokens, attention_weights):
    """
    Generate a diagram representing the self-attention scores for a single
    attention head. The diagram shows one row and column for each of the
    `tokens`, and cells are shaded based on `attention_weights`, with lighter
    cells corresponding to higher attention scores.
    The diagram is saved with a filename that includes both the `layer_number`
    and `head_number`.
    """
    # Create new image
    image_size = GRID_SIZE * len(tokens) + PIXELS_PER_WORD
    img = Image.new("RGBA", (image_size, image_size), "black")
    draw = ImageDraw.Draw(img)
    # Draw each token onto the image
    for i, token in enumerate(tokens):
        # Draw token columns
        token_image = Image.new("RGBA", (image_size, image_size), (0, 0, 0, 0))
        token_draw = ImageDraw.Draw(token_image)
        token_draw.text(
            (image_size - PIXELS_PER_WORD, PIXELS_PER_WORD + i * GRID_SIZE),
            token,
            fill="white",
            font=FONT
        )
        token_image = token_image.rotate(90)
        img.paste(token_image, mask=token_image)
```

```
        # Draw token rows
        _, _, width, _ = draw.textbbox((0, 0), token, font=FONT)
        draw.text(
            (PIXELS_PER_WORD - width, PIXELS_PER_WORD + i * GRID_SIZE),
            token,
            fill="white",
            font=FONT
        )
    # Draw each word
    for i in range(len(tokens)):
        y = PIXELS_PER_WORD + i * GRID_SIZE
        for j in range(len(tokens)):
            x = PIXELS_PER_WORD + j * GRID_SIZE
            color = get_color_for_attention_score(attention_weights[i][j])
            draw.rectangle((x, y, x + GRID_SIZE, y + GRID_SIZE), fill=color)
    # Save image
    img.save(f"Attention_Layer{layer_number}_Head{head_number}.png")
if __name__ == "__main__":
    main()
```

```
# Analysis
## Layer 3, Head 1
Tokens appear to attend strongly to the token that immediately precedes them
in the sequence, suggesting this head captures local positional relationships.
Example Sentences:
- "The dog barked loudly at the stranger."
- "The chef prepared a delicious meal for the guests."
## Layer 5, Head 4
Pronouns appear to attend strongly to related pronouns and nouns nearby,
suggesting this head captures coreference relationships. For example, "she"
and "her" attend to each other, indicating the model links pronouns to their
referents.
Example Sentences:
- "She did her homework before dinner."
- "The dog barked loudly at the stranger."
```

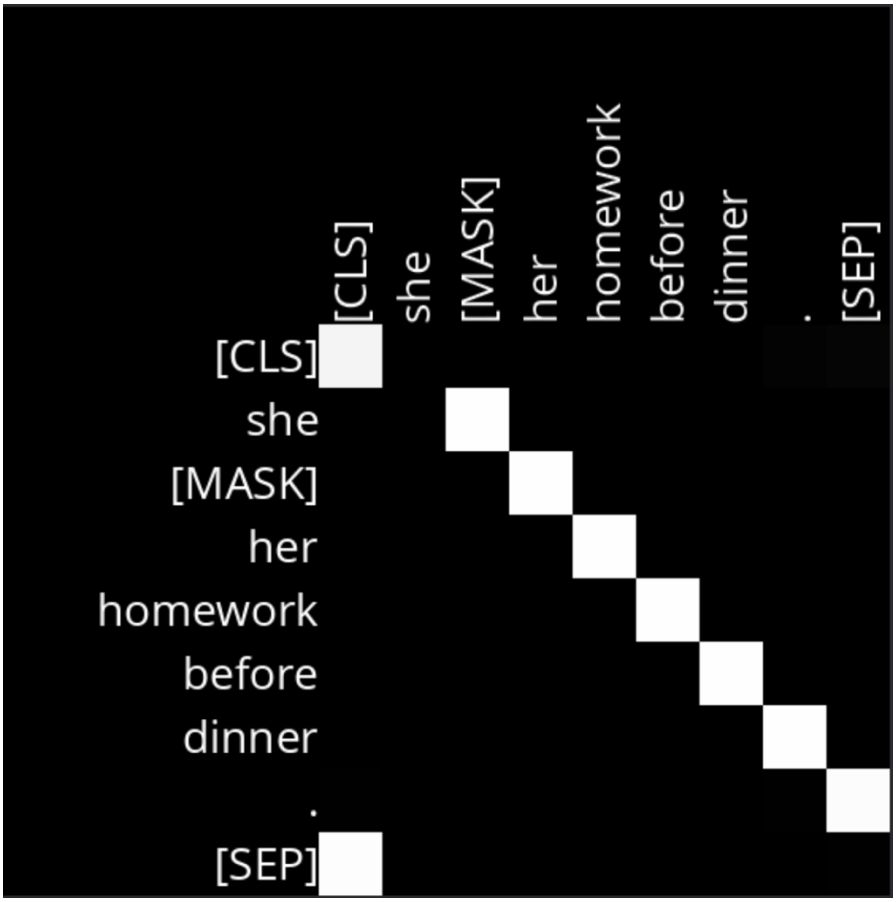


Figure 14 - Attention | Result | Layer 3, Head 1



Figure 15 - Attention | Result | Layer 5, Head 4

Professional Skilss

360º Feedback

As part of the professional development component of this period, I requested feedback from peers and colleagues using a structured 360° feedback form. The responses below reflect how others perceive me in terms of personal qualities, working style, and areas for growth.

Réka Szunyogh-Kocsis, Friend

Réka describes me as a hard-working and honest person who is kind and easy to be around. She identifies honesty as my biggest strength, noting that while many people have valuable inputs but never speak up, I always tell the truth, which helps solve problems and clarify situations efficiently. She highlights that this strength is not tied to a single moment but is consistently present in how I show up for the people around me.

Jeet, Classmate

Jeet describes me as hardworking and straightforward, always on task and focused. He sees my role in a team as a general contributor who brings ideas and helps move things forward. As a concrete example, he recalls a treasure hunt activity where we worked together and I stayed focused on answering questions about the search algorithm. He mentions that he would come to me for general thinking on how to approach a problem.

Joana, HighSchool Friend

Joana describes me as reliable, determined, and easy to work with. She highlights loyalty, adaptability, and a problem-solving mindset as my main strengths, noting that I stay practical and focused under pressure instead of overcomplicating things. She recalls a stressful team situation where I stayed calm, helped organise what needed to be done, and supported everyone involved, showing both leadership and reliability. As a tip, she suggests that because I tend to be very action-oriented, I could benefit from delegating more and trusting others with tasks. She also recommends continuing to develop technical expertise alongside the soft skills I already have, and points to leadership, communication, and staying updated with emerging technologies as areas worth investing in.

Marta Arrais, Highschool Classmate

Marta describes me as someone who is fun and relaxed socially, but extremely organised, hardworking, and focused when it is time to get things done. She highlights organisational skills, strategic planning, time management, and the ability to break complex tasks into manageable steps as my key strengths. She recalls group projects from high school where I always prepared everything ahead of time, ensuring deadlines were met and reducing stress for the whole team. As areas for improvement, she suggests working on active listening and being more open to alternative approaches from teammates, noting that my determination and structured mindset can sometimes lead me to overlook other people's ideas. For the future, she recommends combining my organisational strengths with a stronger focus on understanding end-user needs.

India, Friend

India describes me as outgoing, friendly, and genuinely nice, with a sense of humour that lifts the mood in the team. She highlights strong communication skills, an approachable personality, and a high level of honesty as my main strengths. As a concrete example, she recalls a stressful project situation where I kept the environment positive, stepped up to support a teammate struggling with coding, and gave honest feedback that helped the team fix a broken feature quickly. As areas for improvement, she suggests ensuring that unfiltered feedback is always delivered with empathy, actively inviting quieter team members to speak, and balancing social energy with focused independent work time. For the future, she recommends developing code-review etiquette for delivering direct feedback constructively, active listening during technical discussions, and soft skills for managing non-technical stakeholders.

Julian, Classmate

Julian describes me as patient, detail-oriented, and someone who works well in teams. He identifies consistency and work ethic as my key strengths, noting that I consistently show up, deliver work on time, and make sure it meets quality standards. He mentions that he would come to me to have tasks or computer science concepts explained. He sees no significant room for improvement as a future IT specialist, and recommends simply continuing on the current path, trusting that skills and competences will grow naturally.

Self Reflection

Reflecting on my collaboration competence across the three dimensions of Knowledge, Skills and Attitude, I see the following. In terms of knowledge, I bring a solid foundation in Python programming and AI concepts, which are central to Project Innovate. As a PDP goal, I aim to deepen my understanding of system architecture and how to translate technical knowledge into well-documented, maintainable solutions that others can build on. In terms of skills, my strongest contributions are problem decomposition, structured planning, and staying focused under pressure, qualities that help keep projects on track and give teammates a clear direction to follow. In terms of attitude, I am reliable, honest, and determined. I show up consistently, take ownership of what I commit to, and tend to stay calm and solution-focused in stressful situations.

Comparing Feedback with Self-Image

Comparing the feedback with my own self-image, there is strong consistency around the qualities of being hardworking, focused, and reliable. The honesty that Réka highlights and the calm under pressure that Joana describes are also things I recognise in myself. However, one discrepancy stood out: both Marta and Joana pointed out that my determination and action-oriented approach can sometimes make me less open to others' ideas, or cause me to take on too much alone instead of delegating. This is not something I had fully acknowledged before. I tend to move quickly and trust my own structured plan, but this feedback made me realise that doing so can close off alternatives that could actually improve the outcome. This was a valuable new insight that I intend to carry into future collaboration.

Learning Points

My learning point for future collaboration is to practise active listening during team discussions, letting others finish their ideas before responding or redirecting. When working in a group, I will consciously pause before taking over a task and ask whether it could be shared or delegated instead. This will help me collaborate more effectively while still contributing my organisational strengths.

FeedForward, Feed Up and More

Assignment 1, Essay

Two professional skills that I think are especially relevant for a Computer Science student are communication and teamwork. They might not be the first things that come to mind when thinking about IT, but in practice they matter just as much as the technical side.

Communication in a professional context means being able to share ideas and decisions clearly, whether that is with teammates, lecturers, or clients. In IT this shows up constantly, writing documentation, explaining a design choice, or presenting a solution. Knowing how something works is one thing, but being able to explain it to someone else is a completely different challenge.

For me personally, this is an area I still need to work on. I am comfortable working through technical problems on my own, but putting my reasoning into words that others can follow does not always come naturally. Writing the descriptions for each assignment in this portfolio has actually been useful practice for this, it forces me to explain not just what the code does, but why I made certain decisions.

Teamwork is about being able to work effectively with others towards a shared goal. This includes contributing your part, supporting teammates when things get difficult, and being open to approaches that are not your own. In software development this is basically unavoidable, almost nothing gets built alone.

In terms of where I stand, I think I am a reliable and organised team member who helps keep things on track. But feedback I received this period pointed out that I can sometimes be too focused on my own plan, which means I do not always leave enough room for other people's ideas. That is something I am actively trying to change, listening more before jumping into action, and delegating instead of just doing everything myself.

Both skills feed into each other. If you communicate well, teamwork becomes easier. And working in a team constantly forces you to practise communication. For anyone going into IT, getting better at both is not really optional.

Assignment 2, Pair Assignment

My team has been facing difficulties keeping everyone on track when several members are going through hard times simultaneously. To address this, we proposed holding bi-weekly check-in meetings at the start and end of each week, where everyone shares how much they can contribute that week and whether they are facing any issues. This allows the team to redistribute tasks early and avoid missed deadlines. Alongside this, we suggested using a task-tracking tool such as a whiteboard to make responsibilities visible and create a shared sense of accountability. When a team member is struggling, the focus should shift to finding a practical solution rather than placing pressure on the individual, asking "How can we get this task done this week?" keeps the team moving forward while making the person in difficulty feel supported rather than judged.

Assignment 3, Individual Reflection

As a Computer Science student, I often focus on getting the technical side right, making the code work, passing the tests, submitting on time. However, one area where I consistently fall short is communicating my ideas clearly. Whether it is explaining design choices, writing documentation, or presenting a solution to peers, I struggle to translate what I understand technically into language others can follow. This is where I need feedforward the most.

The challenge is that I tend to work through uncertainty alone. While independence has its value, it also means I miss out on early input that could shape my thinking before habits solidify. To ask for feedforward more effectively, I need to change this habit deliberately. Rather than waiting until I have a finished product, I can approach lecturers or peers at the planning or drafting stage with specific questions, such as whether my explanation of an algorithm makes sense to someone unfamiliar with it. Specific questions invite specific, useful responses.

In group work, feedforward becomes even more valuable. I aim to share early drafts of any written or presented components I contribute and explicitly invite critique before submission. I can also model this behaviour by offering structured feedforward to teammates, focusing not on what went wrong but on what could be stronger next time. This way, feedforward becomes a shared team habit rather than a one-sided request, improving both my communication skills and our collective output.

Self-Reflection, 3rd Period

Looking back on this semester, I can say with confidence that it was a period of genuine growth, not because everything came easily, but because I approached every assignment with curiosity and a real desire to understand what I was building.

The most challenging part of the semester was undoubtedly the work involving artificial intelligence. Projects like Tic-Tac-Toe and Knights required a different kind of thinking, moving away from straightforward input/output logic and towards reasoning about game states, logical implications, and recursive algorithms. The Tic-Tac-Toe assignment in particular demanded the most time and effort. Implementing the Minimax algorithm correctly, ensuring it evaluated all possible game states and always returned the optimal move, was a process that pushed me to think more carefully about recursion and abstract problem-solving than any previous assignment had.

On the Embedded Systems side, this was already my third period working with hardware, so the components themselves were familiar. What made this semester different was the complexity of the projects. The one I am most proud of is the Attendance System, combining RFID, a real-time clock, an LCD display, and a web server into a single cohesive project felt like a genuine engineering challenge, and seeing it work as intended was very rewarding.

What I noticed throughout both areas is that nothing truly surprised me, not because the work was easy, but because I stayed consistent. I worked steadily through each week with focus and genuine interest, which meant challenges were met as they came rather than becoming obstacles. That consistency is something I recognise as a strength in my way of working: I maintain a good method of organising both tasks and time, and that kept me on track throughout the semester.

If I had to identify something to improve, it would be pushing myself further into the areas that challenge me most, particularly AI, rather than consolidating what I already know well. The foundation is solid; the next step is to be more ambitious with it.

Self-Reflection, 4th Period

This period was different from previous ones in an interesting way. For the first time, there were no assignment card with fixed deadlines. In practice there was still an expected pace, each week had its set of tasks, but it was up to each of us to organise ourselves and get things done without the usual external pressure of a hard deadline. I found this a worthwhile challenge in terms of self-management.

In terms of work, I enjoyed having the opportunity to develop the solo project further, now with the added requirement of integrating AI, and to finally move the group project from concept to reality. Last period it was just an idea, but this period we actually had to build it.

I would not say I felt a major evolution this period, more that I was able to handle the work well and keep up with what was expected. One thing I did notice though is that my writing became more concise. I got better at saying the same thing in fewer words, which is something I will carry forward.

Towards the end of the period I went through a difficult time on a personal level, which had some impact on my contribution to the group project. Fortunately it happened late enough that the consequences were limited, but it was a reminder that personal circumstances can affect team dynamics in ways that are hard to anticipate.

Links

Assignements

link: https://github.com/franciscpime/CS_4th_Period

Solo Project

link: https://github.com/franciscpime/math_animator

Group Project

link: https://github.com/Fumbre/potatao